

I'll help you configure this application for Netlify deployment. Since Netlify is primarily a frontend hosting platform, we need to adapt our architecture. Here's the complete setup:

Updated Architecture for Netlify

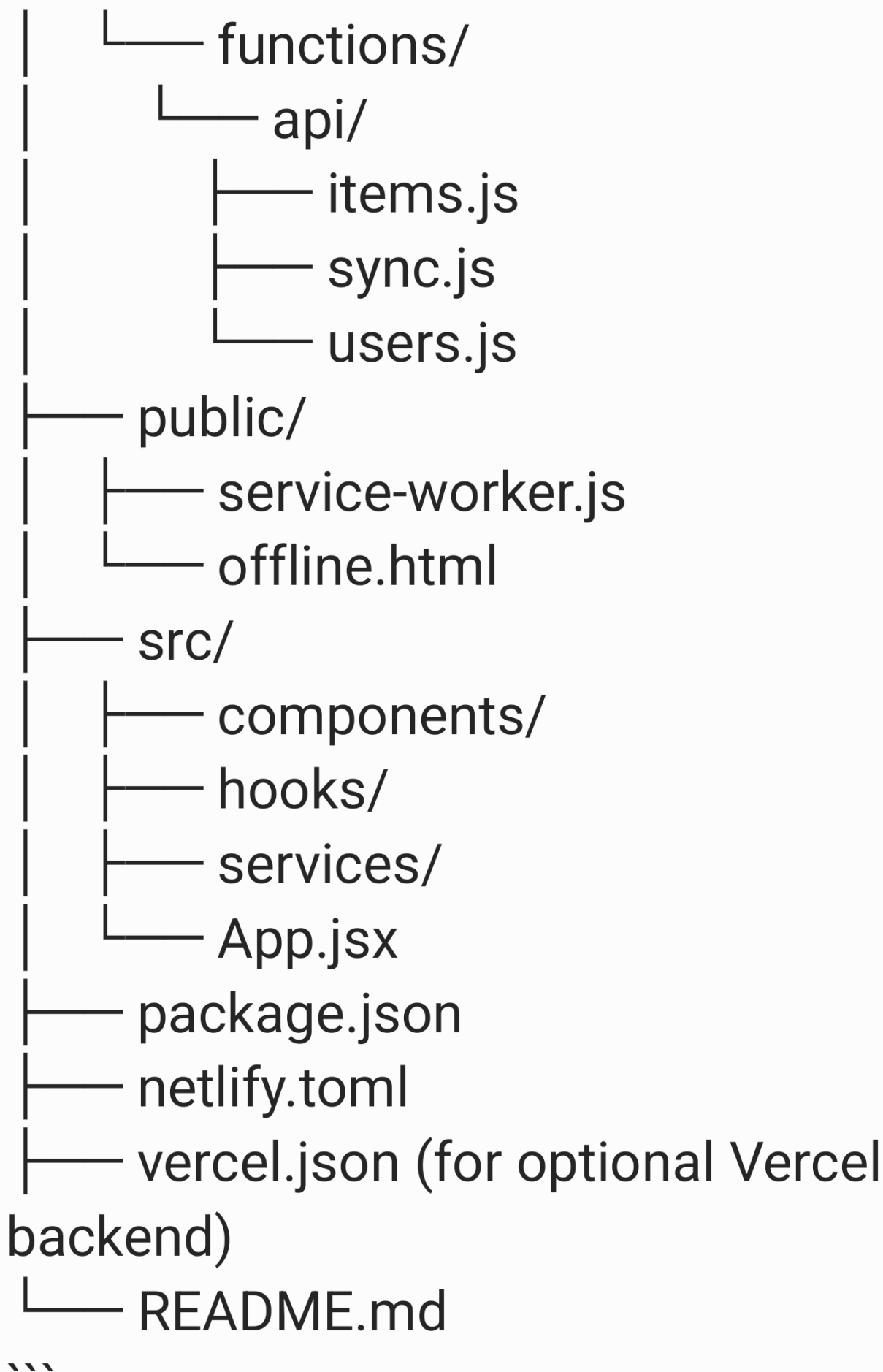
We'll use:

1. Frontend: Hosted on Netlify
2. Backend: Serverless functions (Netlify Functions) + MongoDB Atlas
3. Database: MongoDB Atlas (cloud database)

Updated Project Structure

...

```
offline-first-app/  
└── netlify/
```



1. Netlify Configuration

netlify.toml (Root)

```
``toml
```

```
[build]
```

```
  base = "."
```

```
  publish = "build"
```

```
  command = "npm run build"
```

```
[build.environment]
```

```
  NODE_VERSION = "18"
```

```
  REACT_APP_API_URL = "/.netlify/  
functions/api"
```

```
[[redirects]]
```

```
  from = "/api/*"
```

```
  to = "/.netlify/functions/api/:splat"
```

```
  status = 200
```

```
[[redirects]]
```

```
  from = "/*"
```

to = "/index.html"

status = 200

[[headers]]

for = "/*"

[headers.values]

X-Frame-Options = "DENY"

X-XSS-Protection = "1; mode=block"

X-Content-Type-Options = "nosniff"

Referrer-Policy = "strict-origin-when-cross-origin"

[[headers]]

for = "/service-worker.js"

[headers.values]

Cache-Control = "no-cache, no-store, must-revalidate"

Service-Worker-Allowed = "/"

Functions configuration

[functions]

```
node_bundler = "esbuild"  
directory = "netlify/functions"
```

```
# Environment variables (set in Netlify  
dashboard)
```

```
[context.production.environment]  
  MONGODB_URI = "@mongodb_uri"  
  JWT_SECRET = "@jwt_secret"  
  NODE_ENV = "production"
```

```
[context.deploy-preview.environment]  
  MONGODB_URI = "@mongodb_uri_dev"  
  JWT_SECRET = "@jwt_secret_dev"  
  NODE_ENV = "development"  
````
```

## 2. Serverless Backend Functions

```
netlify/functions/api/_shared/mongodb.js
```

```
````javascript
```

```
import { MongoClient } from 'mongodb';
```

```
const MONGODB_URI =
```

```
process.env.MONGODB_URI;
```

```
const MONGODB_DB = 'offlineapp';
```

```
if (!MONGODB_URI) {
```

```
  throw new Error('Please define the  
MONGODB_URI environment variable');  
}
```

```
let cachedClient = null;
```

```
let cachedDb = null;
```

```
export async function
```

```
connectToDatabase() {
```

```
  if (cachedClient && cachedDb) {  
    return { client: cachedClient, db:  
cachedDb };  
  }
```

```
const client = await
MongoClient.connect(MONGODB_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
  maxPoolSize: 10,
});
```

```
const db = client.db(MONGODB_DB);
```

```
cachedClient = client;
```

```
cachedDb = db;
```

```
return { client, db };
```

```
}
```

```
// Schema definitions
```

```
export const collections = {
```

```
  ITEMS: 'items',
```

```
  USERS: 'users',
```

```
  SYNC_QUEUE: 'syncQueue',
```

```
  SESSIONS: 'sessions'
```

```
};
```

```
// Helper functions
```

```
export async function
```

```
withDatabase(handler) {
```

```
  try {
```

```
    const { db } = await
```

```
connectToDatabase();
```

```
    return handler(db);
```

```
  } catch (error) {
```

```
    console.error('Database connection  
error:', error);
```

```
    throw error;
```

```
  }
```

```
}
```

```
``
```

```
netlify/functions/api/_shared/auth.js
```

```
``javascript
```

```
import jwt from 'jsonwebtoken';
```

```
import crypto from 'crypto';

const JWT_SECRET =
process.env.JWT_SECRET || 'your-secret-
key-change-in-production';
const JWT_EXPIRES_IN = '7d';

export function generateToken(userId) {
  return jwt.sign({ userId }, JWT_SECRET, {
    expiresIn: JWT_EXPIRES_IN });
}

export function verifyToken(token) {
  try {
    return jwt.verify(token, JWT_SECRET);
  } catch (error) {
    return null;
  }
}

export function hashPassword(password) {
```

```
    return  
    crypto.createHash('sha256').update(password + JWT_SECRET).digest('hex');  
}
```

```
export function authenticate(handler) {  
    return async (event, context) => {  
        try {  
            const token =  
event.headers.authorization?.replace('Bearer ', '');  
  
            if (!token) {  
                return {  
                    statusCode: 401,  
                    body: JSON.stringify({ error: 'No  
token provided' })  
                };  
            }  
        }  
    }  
}
```

```
    const decoded = verifyToken(token);
```

```
if (!decoded) {  
  return {  
    statusCode: 401,  
    body: JSON.stringify({ error: 'Invalid  
token' })  
  };  
}
```

```
// Add user to context  
context.userId = decoded.userId;
```

```
return handler(event, context);  
} catch (error) {  
  return {  
    statusCode: 500,  
    body: JSON.stringify({ error:  
'Authentication failed' })  
  };  
}  
};
```

```
}  
...  

```

netlify/functions/api/items.js

```
```javascript  
import { withDatabase, collections } from './
_shared/mongodb.js';
import { authenticate } from './_shared/
auth.js';
import { v4 as uuidv4 } from 'uuid';

// Helper to build response
function buildResponse(statusCode, body)
{
 return {
 statusCode,
 headers: {
 'Content-Type': 'application/json',
 'Access-Control-Allow-Origin': '*',
 'Access-Control-Allow-Headers':
```

```
'Content-Type, Authorization',
 'Access-Control-Allow-Methods': 'GET,
POST, PUT, DELETE, OPTIONS'
},
 body: JSON.stringify(body)
};
}
```

```
export const handler = authenticate(async
(event, context) => {
 // Handle CORS preflight
 if (event.httpMethod === 'OPTIONS') {
 return buildResponse(200, {});
 }
}
```

```
try {
 const { userId } = context;

 switch (event.httpMethod) {
 case 'GET':
 return await getItems(event, userId);
 }
}
```

```
case 'POST':
 return await createItem(event, userId);
case 'PUT':
 return await updateItem(event, userId);
case 'DELETE':
 return await deleteItem(event, userId);
default:
 return buildResponse(405, { error:
'Method not allowed' });
}
} catch (error) {
 console.error('Error in items handler:',
error);
 return buildResponse(500, { error:
'Internal server error' });
}
});
```

```
async function getItems(event, userId) {
 return withDatabase(async (db) => {
 const {
```

```
status,
sort = '-createdAt',
page = 1,
limit = 50,
lastSync
} = event.queryStringParameters || {};
```

```
const query = {
 userId,
 status: { $ne: 'deleted' },
 ...(status && { status })
};
```

```
if (lastSync) {
 query.lastModified = { $gt: new
Date(lastSync) };
}
```

```
// Parse sort
const sortObj = {};
if (sort.startsWith('-')) {
```

```
 sortObj[sort.substring(1)] = -1;
 } else {
 sortObj[sort] = 1;
 }
```

```
const items = await
db.collection(collections.ITEMS)
 .find(query)
 .sort(sortObj)
 .skip((parseInt(page) - 1) *
parseInt(limit))
 .limit(parseInt(limit))
 .toArray();
```

```
const total = await
db.collection(collections.ITEMS).countDoc
uments(query);
```

```
return buildResponse(200, {
 items,
 pagination: {
```

```
 total,
 page: parseInt(page),
 limit: parseInt(limit),
 pages: Math.ceil(total / limit)
 }
});
});
}
```

```
async function createItem(event, userId) {
 return withDatabase(async (db) => {
 const body = JSON.parse(event.body ||
'{}');
 const { clientId, offlineOperation = false }
= body;
```

```
 // Generate clientId if not provided
 const finalClientId = clientId || `local_${
{Date.now()}_${uuidv4()}`;
```

```
 // Check for conflicts
```

```
const existingItem = await
db.collection(collections.ITEMS).findOne({
 userId,
 clientId: finalClientId
});
```

```
const now = new Date();
const item = {
 ...body,
 userId,
 clientId: finalClientId,
 status: body.status || 'pending',
 createdAt: now,
 lastModified: now,
 syncStatus: 'synced',
 offlineOperation,
 _id: uuidv4()
};
```

```
if (existingItem) {
 // Update existing
```

```
 await
db.collection(collections.ITEMS).updateOne(
 { _id: existingItem._id },
 { $set: item }
);
} else {
 // Insert new
 await
db.collection(collections.ITEMS).insertOne(
 item);
}

return buildResponse(201, item);
});
}
```

```
async function updateItem(event, userId) {
 return withDatabase(async (db) => {
 const { id } = event.pathParameters;
 const updates = JSON.parse(event.body
```

```
|| '{}');
```

```
 const result = await
db.collection(collections.ITEMS).findOneAndUpdate(
 { _id: id, userId },
 {
 $set: {
 ...updates,
 lastModified: new Date()
 }
 },
 { returnDocument: 'after' }
);
```

```
 if (!result.value) {
 return buildResponse(404, { error: 'Item
not found' });
 }
```

```
 return buildResponse(200, result.value);
```

```
});
}
```

```
async function deleteItem(event, userId) {
 return withDatabase(async (db) => {
 const { id } = event.pathParameters;

 const result = await
db.collection(collections.ITEMS).findOneAndUpdate(
 { _id: id, userId },
 {
 $set: {
 status: 'deleted',
 lastModified: new Date()
 }
 }
);

if (!result.value) {
 return buildResponse(404, { error: 'Item
```

```
not found' });
}
```

```
 return buildResponse(200, { message:
 'Item deleted successfully' });
 });
}
``
```

netlify/functions/api/sync.js

```
``javascript
import { withDatabase, collections } from './
_shared/mongodb.js';
import { authenticate } from './_shared/
auth.js';
```

```
function buildResponse(statusCode, body)
{
 return {
 statusCode,
```

```
headers: {
 'Content-Type': 'application/json',
 'Access-Control-Allow-Origin': '*',
},
body: JSON.stringify(body)
};
}
```

```
export const handler = authenticate(async
(event, context) => {
 if (event.httpMethod !== 'POST') {
 return buildResponse(405, { error:
'Method not allowed' });
 }
}
```

```
try {
 const { userId } = context;
 const {
 lastSync,
 pendingOperations = [],
 clientItems = []
```

```
} = JSON.parse(event.body || '{}');
```

```
return withDatabase(async (db) => {
 // Get server changes since last sync
 const serverChanges = await
db.collection(collections.ITEMS)
 .find({
 userId,
 lastModified: { $gt: new
Date(lastSync || 0) }
 })
 .toArray();
```

```
 // Process pending operations
 const processedOps = [];
 const syncQueue =
db.collection(collections.SYNC_QUEUE);
```

```
 for (const op of pendingOperations) {
 try {
 await syncQueue.insertOne({
```

```
 userId,
 operation: op.type,
 collectionName: 'items',
 data: op.data,
 clientId: op.clientId,
 status: 'pending',
 createdAt: new Date(),
 deviceInfo: {
 userAgent: event.headers['user-agent'],
 timestamp: new Date()
 }
});
```

```
processedOps.push({
 clientId: op.clientId,
 success: true
});
} catch (error) {
 processedOps.push({
 clientId: op.clientId,
```

```
 success: false,
 error: error.message
 });
}
}
```

// Process client items that need server  
IDs

```
const serverIds = {};
const itemsCollection =
db.collection(collections.ITEMS);

for (const clientItem of clientItems) {
 if (clientItem.clientId && !
clientItem._id) {
 // This is a new item from client,
assign server ID
 const result = await
itemsCollection.insertOne({
 ...clientItem.data,
 userId,
```

```
 clientId: clientItem.clientId,
 createdAt: new Date(),
 lastModified: new Date(),
 syncStatus: 'synced'
 });
```

```
 serverIds[clientItem.clientId] =
result.insertedId;
 }
}
```

```
return buildResponse(200, {
 serverChanges,
 processedOperations: processedOps,
 serverIds,
 syncTimestamp: new
Date().toISOString(),
 conflicts: []
});
});
} catch (error) {
```

```
 console.error('Sync error:', error);
 return buildResponse(500, { error:
error.message });
 }
});
```
```

netlify/functions/api/users.js

```
```javascript
import { withDatabase, collections } from './
_shared/mongodb.js';
import { generateToken, hashPassword }
from './_shared/auth.js';
import { v4 as uuidv4 } from 'uuid';

function buildResponse(statusCode, body)
{
 return {
 statusCode,
 headers: {
```

```
 'Content-Type': 'application/json',
 'Access-Control-Allow-Origin': '*',
 },
 body: JSON.stringify(body)
};
}
```

```
export const handler = async (event) => {
 if (event.httpMethod === 'OPTIONS') {
 return buildResponse(200, {});
 }
}
```

```
try {
 switch (event.httpMethod) {
 case 'POST':
 const path = event.path.split('/').pop();
 if (path === 'register') {
 return await register(event);
 } else if (path === 'login') {
 return await login(event);
 }
 }
}
```

```
 return buildResponse(404, { error: 'Not
found' });
 default:
 return buildResponse(405, { error:
'Method not allowed' });
 }
} catch (error) {
 console.error('Users error:', error);
 return buildResponse(500, { error:
'Internal server error' });
}
};
```

```
async function register(event) {
 return withDatabase(async (db) => {
 const { email, password, name } =
JSON.parse(event.body || '{}');

 if (!email || !password) {
 return buildResponse(400, { error:
'Email and password required' });
 }
 });
}
```

```
}
```

```
// Check if user exists
```

```
const existingUser = await
db.collection(collections.USERS).findOne({
email });
```

```
if (existingUser) {
 return buildResponse(400, { error: 'User
already exists' });
}
```

```
// Create user
```

```
const userId = uuidv4();
const user = {
 _id: userId,
 email,
 name,
 passwordHash:
hashPassword(password),
 createdAt: new Date(),
 settings: {
```

```
 offlineSync: true,
 syncInterval: 30000
 }
};
```

```
 await
db.collection(collections.USERS).insertOne
(user);
```

```
 // Generate token
 const token = generateToken(userId);

 return buildResponse(201, {
 user: {
 id: userId,
 email: user.email,
 name: user.name
 },
 token
 });
});
```

```
}
```

```
async function login(event) {
 return withDatabase(async (db) => {
 const { email, password } =
 JSON.parse(event.body || '{}');
```

```
 if (!email || !password) {
 return buildResponse(400, { error:
'Email and password required' });
 }
```

```
 const user = await
 db.collection(collections.USERS).findOne({
 email });
```

```
 if (!user || user.passwordHash !==
 hashPassword(password)) {
 return buildResponse(401, { error:
'Invalid credentials' });
 }
```

```
// Generate token
const token = generateToken(user._id);

return buildResponse(200, {
 user: {
 id: user._id,
 email: user.email,
 name: user.name
 },
 token
});
});
}
```

netlify/functions/api/health.js

```
```javascript
export const handler = async () => {
  return {
```

```
statusCode: 200,  
headers: {  
  'Content-Type': 'application/json',  
  'Access-Control-Allow-Origin': '*'  
},  
body: JSON.stringify({  
  status: 'OK',  
  timestamp: new Date().toISOString(),  
  service: 'offline-first-api'  
}))  
};  
};  
...
```

3. Updated Frontend for Netlify

Updated package.json (Root)

```
```json  
{
 "name": "offline-first-netlify",
```

```
"version": "1.0.0",
"private": true,
"dependencies": {
 "react": "^18.2.0",
 "react-dom": "^18.2.0",
 "react-scripts": "5.0.1",
 "dexie": "^3.2.4",
 "axios": "^1.5.0",
 "@tanstack/react-query": "^4.33.0",
 "react-router-dom": "^6.15.0",
 "date-fns": "^2.30.0",
 "uuid": "^9.0.0",
 "@material-ui/core": "^4.12.4",
 "@material-ui/icons": "^4.11.3",
 "mongodb": "^5.7.0",
 "jsonwebtoken": "^9.0.2"
},
"devDependencies": {
 "@netlify/plugin-nextjs": "^5.4.0",
 "netlify-cli": "^15.0.0",
 "esbuild": "^0.19.2",
```

```
"eslint": "^8.48.0"
},
"scripts": {
 "start": "react-scripts start",
 "build": "react-scripts build",
 "test": "react-scripts test",
 "eject": "react-scripts eject",
 "netlify:dev": "netlify dev",
 "netlify:build": "npm run build && netlify
build",
 "netlify:deploy": "netlify deploy --prod"
},
"eslintConfig": {
 "extends": ["react-app"]
},
"browserslist": {
 "production": [">0.2%", "not dead", "not
op_mini all"],
 "development": ["last 1 chrome version",
"last 1 firefox version", "last 1 safari
version"]
}
```

```
}
}
...
```

Updated .env (Root)

```
```env  
REACT_APP_API_URL=/.netlify/functions/  
api  
REACT_APP_VERSION=1.0.0  
REACT_APP_ENABLE_SERVICE_WORKER=t  
rue  
REACT_APP_USE_LOCAL_DB=true  
```
```

Updated src/services/api.js

```
```javascript  
import axios from 'axios';  
import { db } from './database';
```

```
// Use Netlify functions endpoint
const API_BASE_URL =
process.env.REACT_APP_API_URL ||
'/.netlify/functions/api';
```

```
const api = axios.create({
  baseURL: API_BASE_URL,
  timeout: 15000,
  headers: {
    'Content-Type': 'application/json'
  }
});
```

```
// Request interceptor
api.interceptors.request.use(
  (config) => {
    const token =
localStorage.getItem('token');
    if (token) {
      config.headers.Authorization = `Bearer
${token}`;
    }
  }
);
```

```
}
```

```
    // Add offline indicator  
    config.headers['X-Offline-Operation'] = !  
navigator.onLine;  
    config.headers['X-Client-Id'] =  
localStorage.getItem('deviceId') || 'web-  
client';  
  
    return config;  
},  
(error) => Promise.reject(error)  
);
```

```
// Response interceptor  
api.interceptors.response.use(  
  (response) => response,  
  async (error) => {  
    const originalRequest = error.config;  
  
    // Handle offline mode
```

```
if (!navigator.onLine && !
originalRequest._retry) {
    originalRequest._retry = true;

    // Store for background sync
    if (originalRequest.data &&
window?.indexedDB) {
        try {
            const syncData = {
                url: originalRequest.url,
                method: originalRequest.method,
                data: originalRequest.data,
                headers: originalRequest.headers,
                timestamp: Date.now()
            };

            // Store in IndexedDB
            const request =
indexedDB.open('offlineRequests', 1);
            request.onsuccess = function(event)
{
```

```
    const db = event.target.result;
    const tx = db.transaction('requests',
'readwrite');
    const store =
tx.objectStore('requests');
    store.add(syncData);
};
```

```
// Register for background sync
if ('serviceWorker' in navigator &&
'SyncManager' in window) {
    const registration = await
navigator.serviceWorker.ready;
    await
registration.sync.register('sync-requests');
}
} catch (e) {
    console.error('Failed to store offline
request:', e);
}
}
```

```
// Return placeholder response
return Promise.resolve({
  data: {
    message: 'Operation queued for sync',
    offline: true,
    queued: true,
    timestamp: Date.now()
  },
  status: 202
});
}
```

```
// Handle auth errors
if (error.response?.status === 401) {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  window.location.href = '/login';
}
```

```
return Promise.reject(error);
```

```
}  
);
```

```
// API methods
```

```
export const itemAPI = {  
  async getAll(params = {}) {  
    try {  
      const response = await api.get('/items',  
{ params });  
      return response.data;  
    } catch (error) {  
      if (!navigator.onLine) {  
        const items = await db.getAllItems();  
        return {  
          items: items.filter(i => i.status !==  
'deleted'),  
          offline: true,  
          pagination: { total: items.length }  
        };  
      }  
      throw error;  
    }  
  }  
};
```

```
}  
},
```

```
async create(itemData) {  
  const clientId = `local_${Date.now()}_${  
    {Math.random().toString(36).substr(2, 9)}}`;   
  const offlineItem = {  
    ...itemData,  
    clientId,  
    syncStatus: 'pending',  
    lastModified: new Date()  
  };  
};
```

```
// Save locally first  
await db.saveItem(offlineItem);
```

```
try {  
  const response = await api.post('/  
items', {  
    ...itemData,  
    clientId,
```

```
offlineOperation: !navigator.onLine  
});
```

```
if (response.data._id) {  
  await db.updateSyncStatus(clientId,  
'synced', response.data._id);  
}
```

```
return response.data;  
} catch (error) {  
  if (error.response?.data?.queued) {  
    return { ...offlineItem, offline: true,  
queued: true };  
  }  
  throw error;  
}  
},
```

```
async update(id, updates) {  
  try {  
    const response = await api.put(`/items/
```

```
{id}`, updates);  
  return response.data;  
} catch (error) {  
  if (!navigator.onLine) {  
    const localItem = await  
db.getItemByServerId(id);  
    if (localItem) {  
      await db.saveItem({  
        ...localItem,  
        ...updates,  
        syncStatus: 'pending'  
      });  
  
      await db.addToSyncQueue({  
        clientId: localItem.clientId,  
        operation: 'update',  
        collectionName: 'items',  
        data: { serverId: id, ...updates }  
      });  
    }  
    return { message: 'Update queued for
```

```
sync', offline: true };  
    }  
    throw error;  
  }  
},
```

```
async delete(id) {  
  try {  
    const response = await api.delete(`/items/${id}`);  
    return response.data;  
  } catch (error) {  
    if (!navigator.onLine) {  
      const localItem = await  
db.getItemByServerId(id);  
      if (localItem) {  
        await  
db.deleteItem(localItem.clientId);  
      }  
      return { message: 'Delete queued for  
sync', offline: true };  
    }  
  }  
}
```

```
    }  
    throw error;  
  }  
},
```

```
async sync(syncData) {  
  try {  
    const response = await api.post('/sync',  
syncData);  
    return response.data;  
  } catch (error) {  
    console.error('Sync failed:', error);  
    throw error;  
  }  
}  
};
```

```
export const authAPI = {  
  async register(userData) {  
    const response = await api.post('/users/  
register', userData);
```

```
    if (response.data.token) {  
        localStorage.setItem('token',  
response.data.token);  
        localStorage.setItem('user',  
JSON.stringify(response.data.user));  
    }  
    return response.data;  
},
```

```
async login(credentials) {  
    const response = await api.post('/users/  
login', credentials);  
    if (response.data.token) {  
        localStorage.setItem('token',  
response.data.token);  
        localStorage.setItem('user',  
JSON.stringify(response.data.user));  
    }  
    return response.data;  
},
```

```
logout() {  
  localStorage.removeItem('token');  
  localStorage.removeItem('user');  
},  
  
getCurrentUser() {  
  const userStr =  
localStorage.getItem('user');  
  return userStr ? JSON.parse(userStr) :  
null;  
}  
};  
  
export default api;  
````
```

## 4. MongoDB Atlas Setup

### MongoDB Atlas Configuration

1. Create a free MongoDB Atlas account at

<https://www.mongodb.com/cloud/atlas>

2. Create a new cluster (Free tier - M0)

3. Create database user:

- Username: offlineapp
- Password: (generate secure password)

4. Network Access: Add IP address

0.0.0.0/0 (for development) or specific  
Netlify IPs

5. Get Connection String:

```

mongodb+srv://

offlineapp:<password>@cluster0.mongodb
.net/offlineapp?

retryWrites=true&w=majority

```

## Netlify Environment Variables

In Netlify dashboard, go to Site settings >  
Environment variables:

```
```
```

```
MONGODB_URI = mongodb+srv://  
offlineapp:yourpassword@cluster0.mongo  
db.net/offlineapp?  
retryWrites=true&w=majority  
JWT_SECRET = your-super-secret-jwt-key-  
change-this-in-production  
NODE_ENV = production
```

```
```
```

## 5. Deployment Scripts

deploy.js (Optional - for advanced deployment)

```
```javascript  
const { exec } = require('child_process');  
const fs = require('fs');  
const path = require('path');  
  
async function deploy() {
```

```
console.log('🚀 Starting Netlify  
deployment...');
```

```
// Build React app
```

```
console.log('📦 Building React app...');  
await execPromise('npm run build');
```

```
// Deploy to Netlify
```

```
console.log('🚀 Deploying to Netlify...');  
await execPromise('netlify deploy --prod --  
dir=build');
```

```
console.log('✅ Deployment completed!');  
}
```

```
function execPromise(command) {  
  return new Promise((resolve, reject) => {  
    exec(command, (error, stdout, stderr) =>  
    {  
      if (error) {  
        console.error(`❌ Error: ${error}`);  
      }  
    });  
  });  
}
```

```
    reject(error);
    return;
  }
  if (stderr) console.log(` ⚠️ ${stderr}`);
  if (stdout) console.log(` 📄 ${stdout}`);
  resolve(stdout);
});
});
}
```

```
// Run deployment
if (require.main === module) {
  deploy().catch(console.error);
}
````
```

## 6. Service Worker Update for Netlify

Updated public/service-worker.js

```
````javascript
```

```
const CACHE_NAME = 'offline-app-netlify-  
v1';  
const OFFLINE_URL = '/offline.html';  
const API_PREFIX = '/.netlify/functions/  
api';
```

```
// Assets to cache
```

```
const PRECACHE_ASSETS = [  
  '/',  
  '/index.html',  
  '/manifest.json',  
  '/static/js/main.chunk.js',  
  '/static/css/main.chunk.css',  
  '/offline.html',  
  '/icon-192x192.png',  
  '/icon-512x512.png'  
];
```

```
// Install event
```

```
self.addEventListener('install', (event) => {  
  event.waitUntil(  

```

```
    caches.open(CACHE_NAME)
      .then((cache) => {
        console.log('📦 Caching app shell');
        return
cache.addAll(PRECACHE_ASSETS);
      })
      .then(() => self.skipWaiting())
    );
  });

// Activate event
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((cacheNames) => {
      return Promise.all(
        cacheNames.map((cacheName) => {
          if (cacheName !== CACHE_NAME) {
            console.log('🧹 Deleting old cache:',
cacheName);
            return caches.delete(cacheName);
          }
        })
      );
    })
  );
});
```

```
    })  
  );  
  }).then(() => self.clients.claim())  
);  
});
```

// Fetch event - specialized for Netlify
functions

```
self.addEventListener('fetch', (event) => {  
  const { request } = event;  
  const url = new URL(request.url);
```

```
  // Skip non-GET requests
```

```
  if (request.method !== 'GET') {  
    event.respondWith(fetch(request));  
    return;  
  }
```

```
  // Handle Netlify functions
```

```
  if (url.pathname.startsWith(API_PREFIX))  
{
```

```
event.respondWith(networkFirstWithCache  
(request));  
    return;  
}
```

```
// Static assets  
event.respondWith(cacheFirst(request));  
});
```

// Network first with cache fallback for API
async function

```
networkFirstWithCache(request) {  
    const cache = await  
    caches.open(CACHE_NAME);
```

```
    try {  
        // Try network first  
        const networkResponse = await  
        fetch(request);
```

```
// Cache successful responses
if (networkResponse.ok) {
  await cache.put(request,
networkResponse.clone());
}
```

```
return networkResponse;
} catch (error) {
  // Network failed, try cache
  const cachedResponse = await
cache.match(request);
```

```
if (cachedResponse) {
  return cachedResponse;
}
```

```
// If it's an API request and we're offline,
queue for sync
if (request.url.includes(API_PREFIX)) {
  await
queueOfflineRequest(request.clone());
```

```
}
```

```
// Return offline placeholder for HTML  
requests
```

```
if (request.mode === 'navigate') {  
  return caches.match(OFFLINE_URL);  
}
```

```
throw error;
```

```
}
```

```
}
```

```
// Cache first for static assets
```

```
async function cacheFirst(request) {  
  const cachedResponse = await  
caches.match(request);
```

```
if (cachedResponse) {  
  return cachedResponse;  
}
```

```
try {  
    const networkResponse = await  
fetch(request);  
  
    if (networkResponse.ok) {  
        const cache = await  
caches.open(CACHE_NAME);  
        await cache.put(request,  
networkResponse.clone());  
    }  
  
    return networkResponse;  
} catch (error) {  
    // If requesting HTML and offline, show  
offline page  
    if (request.mode === 'navigate') {  
        return caches.match(OFFLINE_URL);  
    }  
  
    throw error;  
}
```

```
}
```

```
// Queue offline requests in IndexedDB
```

```
async function
```

```
queueOfflineRequest(request) {
```

```
  try {
```

```
    const requestData = {
```

```
      url: request.url,
```

```
      method: request.method,
```

```
      headers:
```

```
Array.from(request.headers.entries()).reduc
```

```
e((obj, [key, value]) => {
```

```
  obj[key] = value;
```

```
  return obj;
```

```
}, {}),
```

```
body: await request.text(),
```

```
timestamp: Date.now()
```

```
};
```

```
// Store in IndexedDB
```

```
const db = await openIndexedDB();
```

```
    const tx =  
db.transaction('offlineRequests',  
'readwrite');  
    const store =  
tx.objectStore('offlineRequests');  
    await store.add(requestData);  
  } catch (error) {  
    console.error('Failed to queue offline  
request:', error);  
  }  
}
```

```
// Background sync  
self.addEventListener('sync', (event) => {  
  if (event.tag === 'sync-requests') {  
    event.waitUntil(syncOfflineRequests());  
  }  
});
```

```
async function syncOfflineRequests() {  
  try {
```

```
const db = await openIndexedDB();
const tx =
db.transaction('offlineRequests', 'readonly');
const store =
tx.objectStore('offlineRequests');
const requests = await store.getAll();

for (const requestData of requests) {
  try {
    const response = await
fetch(requestData.url, {
  method: requestData.method,
  headers: requestData.headers,
  body: requestData.body
});

    if (response.ok) {
      // Remove successful request
      const deleteTx =
db.transaction('offlineRequests',
'readwrite');
```

```
        const deleteStore =
deleteTx.objectStore('offlineRequests');
        await
deleteStore.delete(requestData.timestamp)
;
    }
} catch (error) {
    console.error('Sync failed for request:',
error);
}
}
```

```
// Notify clients
const clients = await
self.clients.matchAll();
clients.forEach(client => {
    client.postMessage({
        type: 'SYNC_COMPLETE',
        timestamp: Date.now()
    });
});
```

```
} catch (error) {  
  console.error('Background sync failed:',  
error);  
}  
}
```

// Open IndexedDB

```
function openIndexedDB() {  
  return new Promise((resolve, reject) => {  
    const request =  
indexedDB.open('OfflineStorage', 1);  
  
    request.onupgradeneeded = (event) => {  
      const db = event.target.result;  
  
      if (!  
db.objectStoreNames.contains('offlineReq  
uests')) {  
  
db.createObjectStore('offlineRequests', {  
keyPath: 'timestamp' });  

```

```
}
```

```
};
```

```
    request.onsuccess = (event) =>  
resolve(event.target.result);  
    request.onerror = (event) =>  
reject(event.target.error);  
  });  
}
```

```
// Push notifications
```

```
self.addEventListener('push', (event) => {  
  const options = {  
    body: event.data?.text() || 'New update  
available',  
    icon: '/icon-192x192.png',  
    badge: '/badge-72x72.png',  
    vibrate: [100, 50, 100],  
    data: {  
      url: '/'  
    }  
  }
```

```
};
```

```
event.waitUntil(
```

```
self.registration.showNotification('Offline  
App', options)
```

```
);
```

```
});
```

```
self.addEventListener('notificationclick',  
(event) => {  
  event.notification.close();
```

```
event.waitUntil(
```

```
clients.openWindow(event.notification.data  
?.url || '/')
```

```
);
```

```
});
```

```
...
```

7. Quick Start Guide

Step-by-Step Deployment

1. Create a new repository and push the code:

```
```bash
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/
yourusername/offline-first-netlify.git
git push -u origin main
```
```

1. Connect to Netlify:

- Go to Netlify
- Click "New site from Git"
- Choose your repository

- Build settings:
 - Build command: npm run build
 - Publish directory: build
- Click "Deploy site"

2. Set up MongoDB Atlas:

- Create free cluster
- Get connection string
- Add to Netlify environment variables

3. Configure environment variables in Netlify:

```

```
MONGODB_URI =
your_mongodb_atlas_connection_string
JWT_SECRET = your_jwt_secret_key
```
```

4. Enable serverless functions:

- Netlify automatically detects functions in netlify/functions

5. Configure custom domain (optional):

- Go to Site settings > Domain management

- Add custom domain

Testing Locally

```
```bash
```

```
Install dependencies
```

```
npm install
```

```
Install Netlify CLI globally
```

```
npm install -g netlify-cli
```

```
Set up environment variables locally
```

```
echo "MONGODB_URI=your_mongodb_uri"
```

```
> .env
```

```
echo "JWT_SECRET=your_jwt_secret" >>
```

```
.env
```

```
Start local development
```

```
npm run netlify:dev
```

```
Or run React app only
```

npm start

```

8. Monitoring and Maintenance

Netlify Analytics

- Enable Netlify Analytics in dashboard
- Monitor function invocations and performance
- Set up form submissions (if needed)

MongoDB Atlas Monitoring

- Monitor database performance
- Set up alerts for high usage
- Regular backups (available in paid tiers)

Uptime Monitoring

- Use Netlify's built-in uptime monitoring

- Set up status page at your-site.netlify.app/status

9. Additional Optimizations

vercel.json (For alternative Vercel backend)

```
```json
{
 "functions": {
 "api/*.js": {
 "maxDuration": 10
 }
 },
 "rewrites": [
 {
 "source": "/api/(.*)",
 "destination": "/api/$1"
 },
 {
 "source": "/(.*)",
```

```
 "destination": "/index.html"
 }
]
}
...

```

## Performance Budget (.perf-budget.json)

```
```json
{
  "budgets": [
    {
      "resourceSizes": [
        {
          "resourceType": "script",
          "budget": 200
        },
        {
          "resourceType": "document",
          "budget": 20
        }
      ]
    }
  ]
}

```

```
[,
  "resourceCounts": [
    {
      "resourceType": "third-party",
      "budget": 10
    }
  ]
}
]
```

10. Troubleshooting Common Issues

Functions not working

1. Check Netlify function logs in dashboard
2. Verify MongoDB connection string
3. Check environment variables are set

Service Worker issues

1. Clear site data in browser
2. Check if HTTPS is enabled
3. Verify service worker registration

Database connection issues

1. Check MongoDB Atlas IP whitelist
2. Verify database user permissions
3. Check connection string format

This setup gives you a complete offline-first application deployed on Netlify with serverless backend functions and MongoDB Atlas database. The application will work seamlessly online and offline, with automatic synchronization when connectivity is restored.

